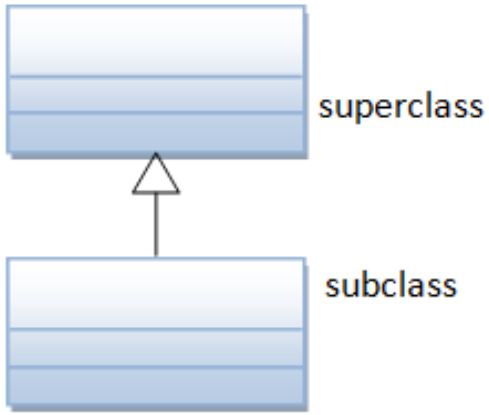


Unit 5

Inheritance

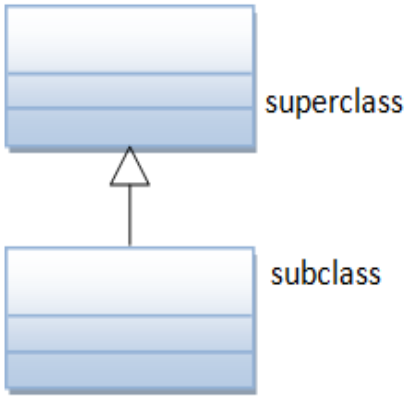


Outline

1. Inheritance Basics
2. Overriding

Inheritance

- **Motivation**
  - Supports the key OOP goal of **code reuse**. Allow us to design **class hierarchies** so that **shared behavior is placed in a super class** then inherited by subclasses (i.e., avoids writing the same code twice to ease maintenance)
- **Ideas**
  - Common attributes and methods are placed in a **superclass** (also called ***parent class*** or ***base class***)
  - You can create a subclass that **inherits** attributes and methods of the super class
    - Subclass also called ***child class*** or ***derived class***

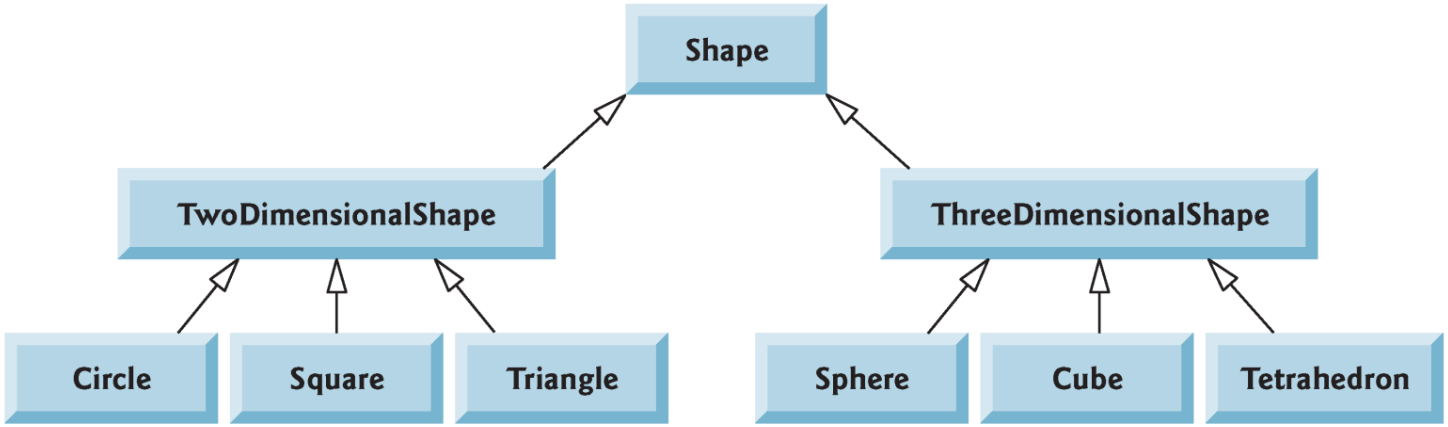


  - Subclass has access to all **non-private** (i.e., *public* and *protected*) **attributes and methods of the superclass**
  - Subclass can also add new attributes/methods and/or overriding the superclass methods
- **Syntax**
  - public class SubClass ***extends*** SuperClass { … }

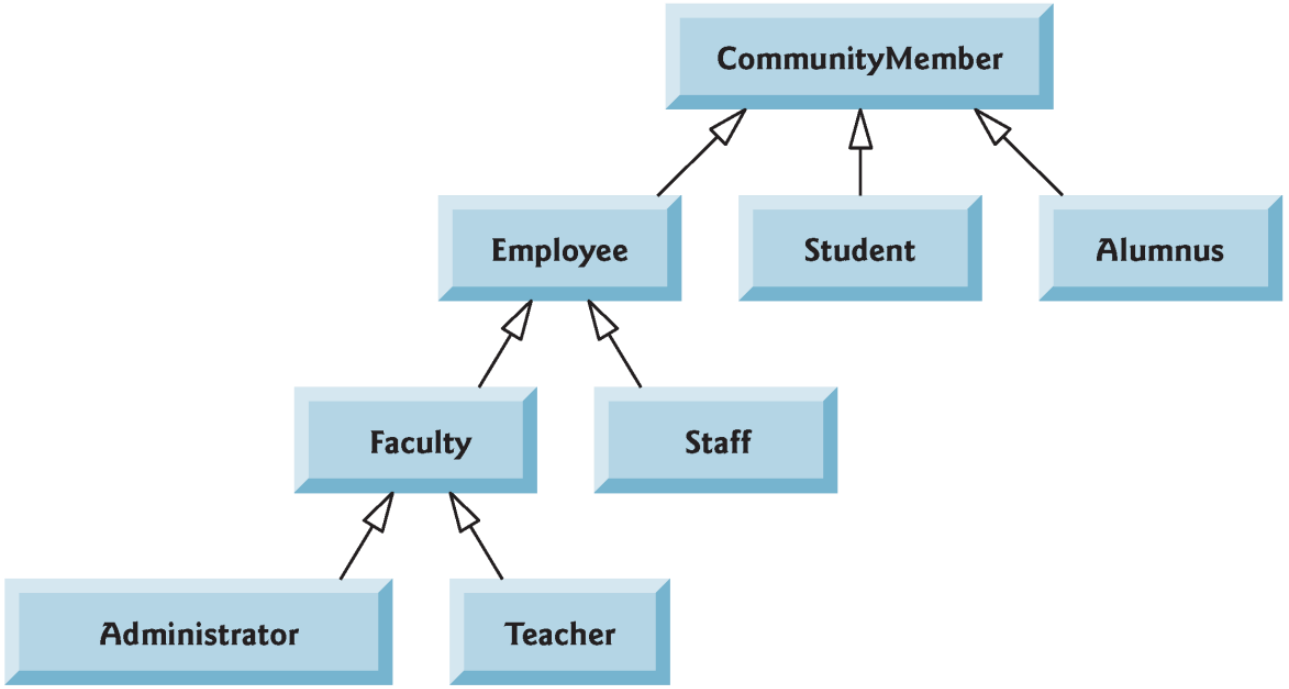
Benefits of Inheritance

- Benefits of inheritance
  - Can save time during program development by basing new classes on existing tested and quality classes.
  - Reduces duplication => eases maintainability of the code
- **Limitation:**
  - Java supports only **single inheritance**, in which each class is derived from exactly one direct superclass

Basic Graphical Example



Basic Graphical Example

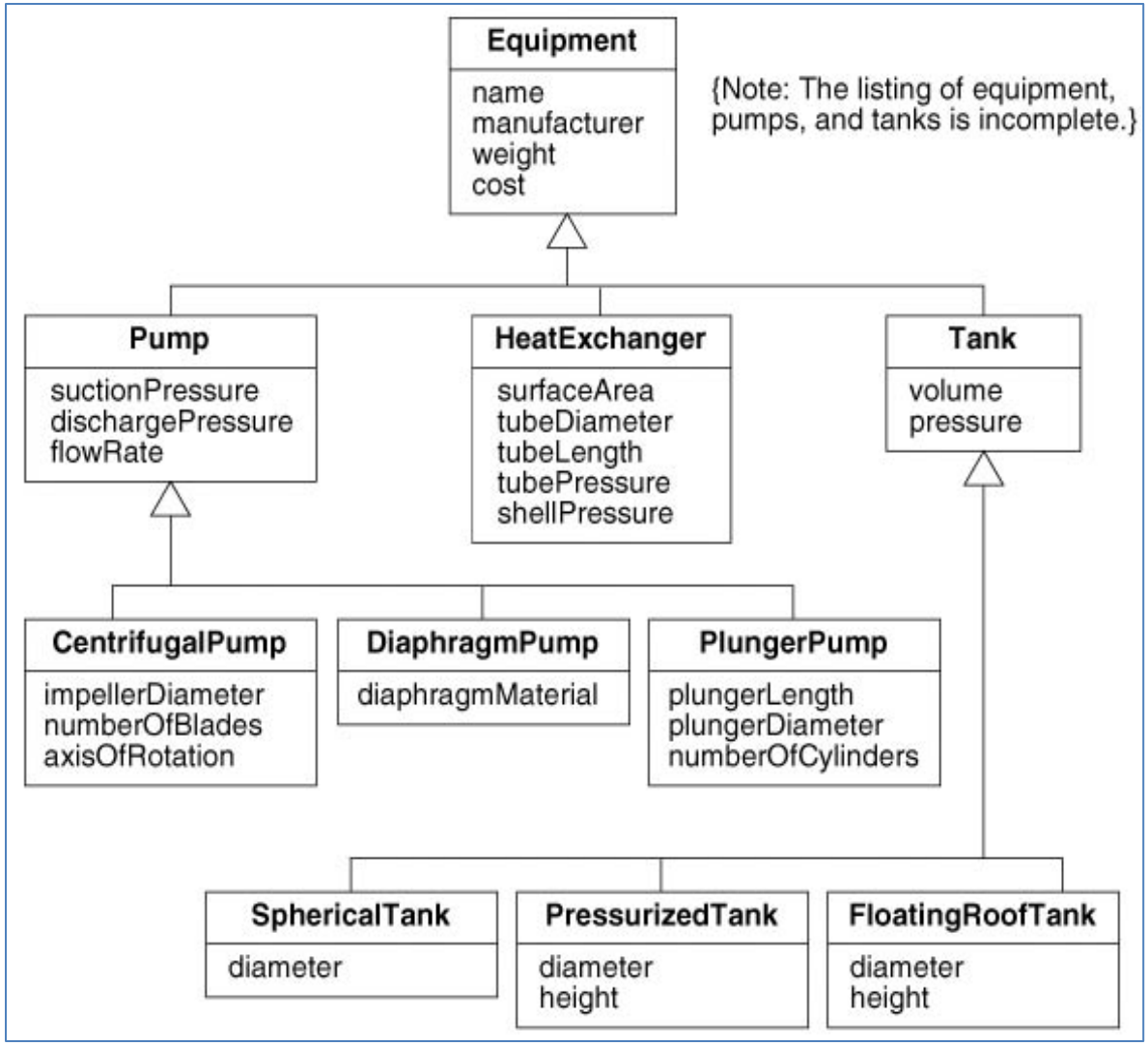


More Examples

Superclass	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

Multilevel Inheritance Hierarchy and Objects

Classes



Objects

P101:DiaphragmPump

name = "P101"
manufacturer = "Simplex"
weight = 100 kg
cost = $5000
suctionPres = 1.1 atm
dischargePres = 3.3 atm
flowRate = 300 l/hr
diaphragmMatl = Teflon

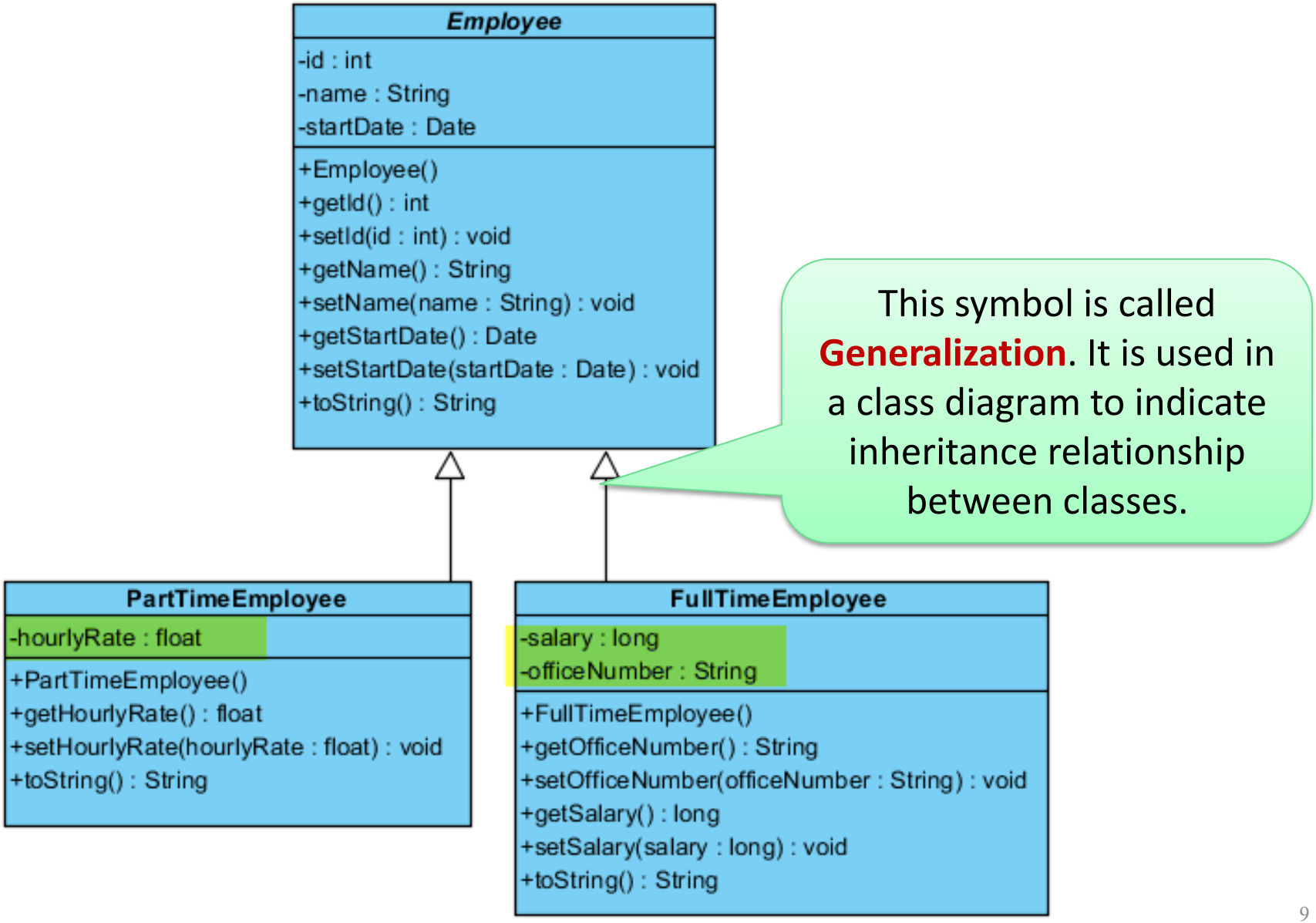
E302:HeatExchanger

name = "E302"
manufacturer = "Brown"
weight = 5000 kg
cost = $20000
surfaceArea = 300 m^2
tubeDiameter = 2 cm
tubeLength = 6 m
tubePressure = 15 atm
shellPressure = 1.7 atm

T111:FloatingRoofTank

name = "T111"
manufacturer = "Simplex"
weight = 10000 kg
cost = $50000
volume = 400000 liter
pressure = 1.1 atm
diameter = 8 m
height = 9 m

Inheritance Example - Employee Hierarchy



Inheritance and Constructors

- **Constructors are not inherited**
- But a subclass constructor **call its direct superclass's constructor** to initialize the instance variables inherited from the superclass.
- Syntax—keyword **super**, followed by a set of parentheses containing the superclass constructor arguments. Must be the first statement in the subclass constructor's body.
- If a subclass constructor does not include an explicit call to the superclass constructor, Java implicitly calls the superclass's default constructor.
- You can explicitly use **super()** to call the superclass's default constructor, but this is rarely done.

Example: Inheritance and Constructors

Inheritance!

Person.java

```
public class Person {
    private String name;

    public Person(String name) {
        this.name = name;
    }
}
```

Customer

```
public class Customer extends Person {
    public String phone;

    public Customer(String n, String p) {
        super(n);
        this.phone = p;
    }
}
```

Employee.java

```
public class Employee extends Person {
    public double salary;

    public Employee(String n, double s) {
        super(n);
        this.salary = s;
    }
}
```

Constructors in Subclasses

- Instantiating a subclass object begins a **chain of constructor calls**
  - The subclass constructor, before performing its own tasks, invokes its direct superclass's constructor
- If the superclass is derived from another class, the superclass constructor invokes the constructor of the next class up the hierarchy all the way back to the Constructor of Object
- The last constructor called in the chain is always class Object's constructor.
- Each superclass's constructor initialized the superclass attributes that the subclass object inherits.

Example: Constructors in Subclasses

Employee.java

```
public class Employee extends Person {
    public double salary;

    public Employee(String n, double s) {
         super(n);
         this.salary = s;
    }

    public double getPayAmount() {
        return this.salary;
    }
}
```

Person.java

```
public class Person {
    private String name;

    public Person(String name)
    {
         this.name = name;
    }
}
```

CommissionEmployee.java

```
public class CommissionEmployee extends Employee {
    double commission;
    int sales;

    public CommissionEmployee(String n, double salary,
                              double commission, int sales) {
        super(n, salary);
        this.commission = commission;
        this.sales = sales;
    }
}
```

Example: ArrayList for Superclass

- Objects of a subclass can be stored in references to a superclass
  - This is commonly done in lists and arrays

```
ArrayList<Person> pList = new ArrayList<Person>();

Customer c = new Customer(“Lionel Messi", “123456789");
pList.add(c);

Employee e = new Employee(“Christiano Ronaldo", 5000);
pList.add(e);

CommissionEmployee ce = new CommissionEmployee(“Donald
Duck", 5000, 0.05, 10000);
pList.add(ce);
```

is-a relationship vs. has-a relationship

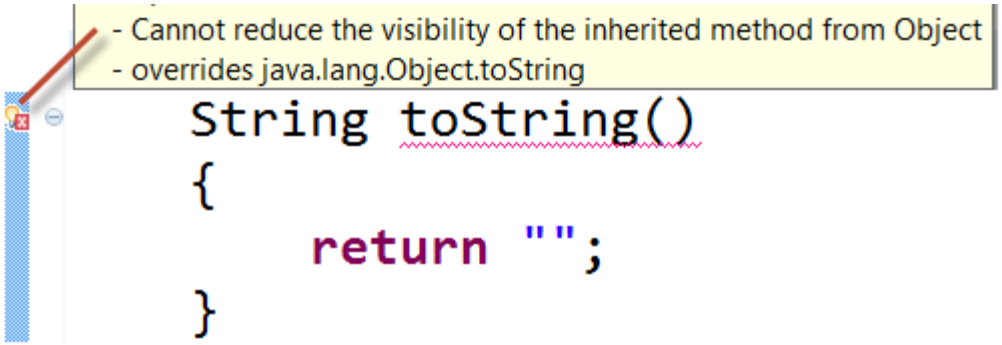
- We distinguish between the is-a relationship and the has-a relationship
- ***Is-a*** represents inheritance
  - In an *is-a* relationship, an object of a subclass can also be treated as an object of its superclass
  - e.g., Student is a Person
- ***Has-a*** represents composition
  - In a *has-a* relationship, an object contains as attributes references to other objects
  - E.g., Student has a list of courses

The Object Class

- Object is the **root** class of all classes in Java
- All other classes are descendents of Object
- **Object** is part of the java.lang package
- Useful Object methods:
  - toString - returns a **string representation** of the object (by default, its class name and id, but this can be overridden).
  - equals - tests for equality of value of two different objects
  - getClass - returns the class to which an object belongs

Overriding

- Overriding = child class redefines the behavior of the parent
- To override a superclass method, a subclass must **declare a method with the same signature as the superclass method**
  - Same access modifier should be used. E.g. if the superclass method is public the overridden method should also be public.



- **@Override is an optional annotation**
  - Declare overridden method with the *@Override* annotation to ensure at compilation time that you defined their signatures correctly.

Example: method override

Employee.java

```
public class Employee extends Person {
    public double salary;

    public Employee(String n, double s) {
        super(n);
        this.salary = s;
    }

    public double getPayAmount() {
        return this.salary;
    }
}
```

Person.java

```
public class Person {
    private String name;

    public Person(String name)
    {
        this.name = name;
    }
}
```

CommissionEmployee.java

```
public class CommissionEmployee extends Employee {
    double commission;
    int sales;

    public CommissionEmployee(String n, double salary,
                              double commission, int sales) {
        super(n, salary);
        this.commission = commission;
        this.sales = sales;
    }

    @Override
    public double getPayAmount() {
        return salary + commission * sales;
    }
}
```

Overriding

- Overriding allow the subclass to replace/extend the behavior of the superclass.
- When a subclass method overrides an inherited superclass method, the superclass method can be accessed from the subclass by preceding the superclass method name with keyword **super** and a dot (.) separator.

```
public class Instructor extends Person {
    private String office;

    public String toString() {
        return super.toString() + " - Office: " + office;
    }
```

instanceof operator

- The **`instanceof`** operator is used to determine if an object is of a particular class.

  `if (shape1 instanceof Circle)`

  Returns **true** if the object to which **shape1** points "is a" **Circle**

- Every object in Java knows its own class by using the **`getClass`** method inherited from the Object class
  - The `getClass` method returns an object of type `Class`
  - To get the object's class name you can use `shape1.getClass().getName()`

Important notes

- It is a **syntax error** to **override** a method with a more restricted access modifier.
  - ✓ A **public** method of the **superclass** cannot become **protected** or **private** in the **subclass**.
- A **final** class **cannot be extended** (cannot be a **superclass**)
- A **final** method **cannot be overridden** (cannot change how it works in a **subclass**)!
- When you **extend** a class, the new class inherits the **superclass's** members—though the private **superclass** members are hidden in the new class.
- To enable a subclass to directly access superclass instance variables, we can declare those members as **protected** in the superclass.
- You can check to see if an object is an instance of a specified class using the **instanceof** operator. It returns either true or false.

Summary

- Inheritance = placing common attributes and methods in a superclass so that subclasses can reuse them
- Subclass extends a superclass:
  - inherit the superclass's members, though the `private` superclass members are hidden from the subclass
  - can define their own additional specialized methods / attributes
  - can override an inherited method
- **Constructors are not inherited** but a subclass constructor **can call its direct superclass's constructor** to initialize the superclass attributes